

Mini Project 4

EECE 7398 Legged Robotics

Code Listing

func_feedback.m

```
1 % Compute control action using feedback linearization
2 %
3 % Inputs:
4 %     x: states
5 %         q1
6 %         q2
7 %         q3
8 %         dq1
9 %         dq2
10 %        dq3
11 %     alpha: Bezier coefficients for q2 and q3
12 %         alpha2 (1st to 5th coefficients)
13 %         alpha3 (1st to 5th coefficients)
14 %     s_params: for gait timing
15 %         q1_min
16 %         delq
17 %
18 % Outputs:
19 %     u: control action
20 %
21 function u = func_feedback(x,alpha,s_params)
22 % gains
23 kp1 = 2500;
24 kp2 = 2500;
25 kd1 = 500;
26 kd2 = 500;
27 % Seperating inputs
28 q = x(1:3); q = q(:);
29 dq = x(4:6); dq = dq(:);
30 % Get model parameters
31 [r,m,Mh,Mt,l,g] = func_model_params;
32 params = [r,m,Mh,Mt,l,g];
33 % Seperate Bezier coefficients
34 alpha2 = alpha(1:5);
35 alpha3 = alpha(6:10);
36 % Seperate s_params (template convention: s_params = [q1_min (post-impact), q1_max (pre-
    impact)])
37 q1_min = s_params(1);
38 q1_max = s_params(2);
39 delq = q1_max - q1_min;
40 % Gait timing variable
41 s = func_gait_timing(q(1), q1_min, q1_max);
42 % Saturate s to [0,1] to prevent Bezier extrapolation
43 if s > 1
44     s = 1;
45 elseif s < 0
46     s = 0;
47 end
48 % Get D,C,G,B matrices
49 [D,C,G,B] = func_compute_D_C_G_B(q,dq,params);
50 % Defining fx and gx (from xdot = f(x) + g(x)*u)
```

```

51 fx = [dq; D\(-C*dq - G)];
52 gx = [zeros(3,2); D\B];
53 % find y = h(x) = q_b - b(s)
54 b2 = bezier(s,4,alpha2);
55 b3 = bezier(s,4,alpha3);
56 h = [q(2) - b2; q(3) - b3];
57 y = h;
58 % Calculating y_dot = Lfh = dh/dx * fx
59 % h is a function of (s, q2, q3, dq), not q1 directly.
60 % Use chain rule: dh/dq1 = dh/ds * ds/dq1 = dh/ds * (1/delq)
61 db_ds2 = d_ds_bezier(s,4,alpha2);
62 db_ds3 = d_ds_bezier(s,4,alpha3);
63 % dh_dx is 2x6: [dh/dq1, dh/dq2, dh/dq3, dh/ddq1, dh/ddq2, dh/ddq3]
64 dh_dx = zeros(2,6);
65 dh_dx(1,1) = -db_ds2/delq; % dh1/dq1 = -db2/ds * 1/delq
66 dh_dx(2,1) = -db_ds3/delq; % dh2/dq1 = -db3/ds * 1/delq
67 dh_dx(1,2) = 1; % dh1/dq2 = 1
68 dh_dx(2,3) = 1; % dh2/dq3 = 1
69 Lfh = dh_dx*fx;
70 dy = Lfh;
71 %%% PD controller
72 Kp = [kp1,0; 0,kp2];
73 Kd = [kd1,0; 0,kd2];
74 v = -Kp*y - Kd*dy;
75 %%% Feedback linearization
76 % dLfh is the 2x6 jacobian of Lfh w.r.t. [q; dq] (with 1/delq on q1 column)
77 dLfh = func_compute_dLfh([s,delq],dq(1),[alpha2,alpha3]);
78 L2fh = dLfh*fx;
79 LgLfh = dLfh*gx;
80 % Control action: u = LgLfh^-1 * (v - L2fh)
81 u = LgLfh\'(v - L2fh);
82 end

```

sim_and_plot_full_dynamics.m

```

1 % Simulates full dynamics for a 3 link biped in the sagittal plane
2 % Needed "inputs" from optimizer:
3 % f = [q10, dq10, alpha(3-5)_q2, alpha(3-5)_q3]
4 % q10: pre-impact initial angle for q1
5 % dq10: pre-impact initial velocity for dq1
6 % alpha(3-5)_q2:
7 % 3rd to 5th Bezier coefficient for q2
8 % alpha(3-5)_q3:
9 % 3rd to 5th Bezier coefficient for q3
10 %
11
12 %-----%
13
14 % Include util and autogen folders
15 set_path
16
17 % Ouput of optimizer
18 % The f vector below is actually walking down a slope - optimizer wasn't
19 % enforcing required constraints
20 % f = [-0.4821, -3.597, 0.2048, 0.1918, 0.3672, 0.6869, 2.3175, 2.2645, 2.8316];
21 % x_max = 0.2048;
22
23
24 % f = [q10, dq10, alpha(3-5)_q2, alpha(3-5)_q3]
25 % q10: pre-impact initial angle for q1
26 % dq10: pre-impact initial velocity for dq1
27 % alpha(3-5)_q2:
28 % 3rd to 5th Bezier coefficient for q2
29 % alpha(3-5)_q3:
30 % 3rd to 5th Bezier coefficient for q3
31 %

```

```

32 f = [-0.4907, -2.3947, 0.3053, 0.6256, 0.8814, 2.6360, 2.7919, 2.9537];
33
34 %------%
35
36 % extract bezier coefficients
37 % MUST match ZD formula exactly - beta1/beta2 feedforward depends on these
38 alpha2 = [-f(5), -f(4), f(3:5)];
39 alpha3 = [f(8)-f(5), -f(7)+2*f(6), f(6:8)];
40
41 a = [alpha2,alpha3];
42
43 % extract preimpact states
44 q1_minus = f(1);
45 dq1_minus = f(2);
46
47 % maximum and minimum angles for q1 during a single gait
48 % Convention matches sim_zero_dynamics: z_min=-f(1)=+0.3065, z_max=f(1)=-0.3065
49 x_max = f(1); % = -0.3065 (pre-impact angle, end of swing)
50 x_min = -f(1); % = +0.3065 (post-impact angle, start of swing)
51 delq = x_max - x_min; % = -0.6130
52
53 %------%
54 %%% Impact map
55 % Need full states to get impact map
56
57 %Mapping from z to x on boundary pg 140, 141
58 q2_minus = alpha2(5);
59 q3_minus = alpha3(5);
60
61 %d_dot- = M*(alpha(M) - alpha(M-1))*theta_dot./(delta_theta)
62 dq2_minus = 4*(alpha2(5)-alpha2(4))*dq1_minus/delq;
63 dq3_minus = 4*(alpha3(5)-alpha3(4))*dq1_minus/delq;
64
65 %I.C. [thetas; velocities]
66 x_minus = [q1_minus, q2_minus, q3_minus, dq1_minus, dq2_minus, dq3_minus];
67
68 % Obtain impact map from pre impact conditions - x_minus
69 % Inputs:
70 %     x_plus: states right before impact
71 %     [q1, q2, q3, dq1, dq2, dq3]
72 %
73 % Output:
74 %     x_plus: states right after impact
75 %     [q1, q2, q3, dq1, dq2, dq3]
76 %     F2: impact forces, x and y components
77 %
78 [x_plus0, ~] = func_impact_map(x_minus);
79
80 % *** Project post-impact body states onto the Bezier manifold (s=0) ***
81 % Physical q3_plus from impact map != b3(0) from Bezier -> 0.59 rad error.
82 % This error generates a large torque that destabilizes the passive q1.
83 % Projecting ensures zero initial tracking error so the controller only
84 % needs to MAINTAIN the manifold, not make a large initial correction.
85 q1_plus = x_plus0(1);
86 dq1_plus = x_plus0(4);
87 x_plus0(2) = bezier(0, 4, alpha2); % q2 on manifold at s=0
88 x_plus0(3) = bezier(0, 4, alpha3); % q3 on manifold at s=0
89 x_plus0(5) = d_ds_bezier(0, 4, alpha2) * dq1_plus / delq; % dq2 consistent with Bezier
90 x_plus0(6) = d_ds_bezier(0, 4, alpha3) * dq1_plus / delq; % dq3 consistent with Bezier
91
92 s_params = [q1_plus, x_max];
93
94 %%% Simulate
95 [t_tot,x_tot,poincare] = sim(x_plus0, a, s_params, x_max, delq);
96
97 %------%
98
99 % Compute and print MCOT

```

```

100 compute_mcot(t_tot, x_tot, poincare, a, x_max);
101
102 % Plots and animation
103
104 plot_trajectories(t_tot,x_tot)
105
106 animate_results(t_tot,x_tot)
107
108 plot_poincare_map(poincare, f)
109
110 %-----%
111
112 % Simulator, wrapped in a function so event function can accept global
113 % variable
114 %
115 % Inputs:
116 %   x_plus0: [q0, dq0]
117 %           Initial conditions for ODE45
118 %   a: [alpha2, alpha3]
119 %       Bezier coefficients for q2 and q3
120 %   s_params: [q1_min, q1_max]
121 %
122 % Outputs:
123 %   t_tot: times (in s) for all steps appended together, time = 0 at the
124 %           begining of each step. This will help to sort steps in plots
125 %   x_tot: [q, dq]
126 %           output of ODE45 for all steps appended together
127 %
128 function [t_tot,x_tot,poincare] = sim(x_plus0, a, s_params, x_max, delq)
129
130 q1_min = s_params(1);
131 q1_max = s_params(2);
132
133 ti = 0; tf = 5;
134
135 options = odeset('Event',@event,'AbsTol',1e-4,'RelTol',1e-4);
136
137 n = 15;      % Number of desired steps the biped should take
138
139 % Define variables where time and states of solution will be appended
140 t_tot = []; x_tot = [];
141 poincare = zeros(n, 2); % post-impact [q1+, dq1+] at each step
142
143 for i = 1:n
144
145     [t,x_sol] = ode45(@(t,x) func_full_dynamics(t,x,a,s_params), [ti, tf], x_plus0, options)
146     ;
147
148     % Apply impact
149     [x_plus, ~] = func_impact_map(x_sol(end,:));
150
151     % Record post-impact [q1+, dq1+] for Poincare map (before projection, q1/dq1 unchanged
152     % by it)
153     poincare(i,:) = [x_plus(1,1), x_plus(1,4)];
154
155     % *** Project body states onto manifold at s=0 (zero initial tracking error) ***
156     alpha2_loc = a(1:5); alpha3_loc = a(6:10);
157     dq1_new = x_plus(1,4); % dq1 post-impact
158     x_plus(1,2) = bezier(0, 4, alpha2_loc); % q2 on manifold
159     x_plus(1,3) = bezier(0, 4, alpha3_loc); % q3 on manifold
160     x_plus(1,5) = d_ds_bezier(0, 4, alpha2_loc) * dq1_new / delq; % dq2 consistent
161     x_plus(1,6) = d_ds_bezier(0, 4, alpha3_loc) * dq1_new / delq; % dq3 consistent
162
163     % New initial condition
164     x_plus0 = x_plus(1,:);
165
166     % Update s_params for next step: new q1_plus becomes q1_min
167     q1_min = x_plus0(1);

```

```

166 q1_max = x_max;
167 s_params = [q1_min, q1_max];
168
169 % Append raw time (starts at 0 each step) - animate_results and
170 % plot_trajectories use find(t==0) to detect step boundaries
171 t_tot = [t_tot; t];
172 x_tot = [x_tot; x_sol];
173
174 fprintf('Step %d: q1_plus=%.4f, dq1_plus=%.4f\n', i, x_plus0(1), x_plus0(4));
175 disp(['Step#...', num2str(i)]);
176
177 end
178
179 %------%
180
181 % Event function - detect when impact happens
182 %
183 % Inputs: [t, x]
184 %
185 % Note: currently using gait timing variable instead of end of swing feet
186 %
187 function [limits, terminal, direction] = event(~, x)
188
189     q = x(1:3);
190     dq = x(4:6);
191
192     s = func_gait_timing(q(1), q1_min, q1_max);
193
194     [r, m, Mh, Mt, l, g] = func_model_params;
195     params = [r, m, Mh, Mt, l, g];
196
197     [~, ~, pm1, pm2, P2] = func_compute_pMh_pMt_pm1_pm2_pcm_P2(q, dq, params);
198     [~, ~, ~, ~, vcm] = func_compute_vMh_vMt_vm1_vm2_vcm(q, dq, params);
199
200     %limits = P2(2) <= 0.01 && pm2(1) > pm1(1) && vcm(2) < 0;
201
202     if s >= 1
203         s = 1;
204     end
205
206     limits = s - 1;
207     terminal = 1;
208     direction = [];
209 end
210
211 end
212
213 %------%
214
215 % Poincare map: step-to-step post-impact convergence
216 %
217 % Inputs:
218 %   poincare: (n x 2) post-impact [q1+, dq1+] per step
219 %   f: unused, kept for call signature compatibility
220 %
221 function plot_poincare_map(poincare, ~)
222
223     n = size(poincare, 1);
224     steps = (1:n-1)';
225
226     % ||z+(k+1) - z+(k)|| across steps
227     delta = vecnorm(diff(poincare), 2, 2);
228
229     figure('Name', 'Poincare Map');
230     plot(steps, delta, 'k-o', 'LineWidth', 1.5, 'MarkerFaceColor', 'k');
231     xlabel('Step number k');
232     ylabel('||z~+(k+1) - z~+(k)||');
233     title('Post-impact Poincar\'e convergence');

```

```

234 grid on;
235
236 end
237
238 %-----%
239
240 % Mechanical Cost of Transport (MCOT)
241 % MCOT = integral(sum_j |u_j * dq_j|) dt / (M_total * g * d_total)
242 %
243 % Inputs:
244 % t_tot: time vector from sim()
245 % x_tot: state matrix from sim() [q1,q2,q3,dq1,dq2,dq3]
246 % poicare: (n x 2) post-impact [q1+, dq1+] -- gives q1_min per step
247 % a: [alpha2, alpha3] Bezier coefficients
248 % x_max: pre-impact q1 (q1_max, same every step)
249 %
250 function compute_mcot(t_tot, x_tot, poicare, a, x_max)
251
252 [r, m, Mh, Mt, l, g] = func_model_params;
253 M_total = 2*m + Mh + Mt;
254 params = [r, m, Mh, Mt, l, g];
255
256 % Detect step boundaries (t resets to 0 at start of each ODE45 solve)
257 ind0 = find(t_tot == 0);
258 n = length(ind0);
259 ind0(end+1) = length(t_tot) + 1; % sentinel
260
261 J_sum = 0; % sum of norm(u)^2 across all timesteps
262 N_total = 0; % total number of timesteps
263 d_total = 0; % total distance walked
264 t_sim_total = 0; % total simulation time
265
266 for i = 1:n
267     idx = ind0(i) : ind0(i+1)-1;
268     t_step = t_tot(idx);
269     x_step = x_tot(idx, :);
270
271     % s_params: q1_min = post-impact q1 recorded at the start of this step
272     s_p = [poicare(i,1), x_max];
273
274     % Accumulate sum(norm(u)^2) -- same formula as optimizer cost J
275     for j = 1:length(t_step)
276         z = x_step(j, [1 4]); % [q1, dq1]
277         u = func_compute_control_action(z, a, s_p);
278         J_sum = J_sum + norm(u)^2;
279     end
280     N_total = N_total + length(t_step);
281
282     % Step duration and distance -- use same P2(1) formula as animate_results
283     t_sim_total = t_sim_total + t_step(end);
284     [~,~,~,~, P2] = func_compute_pMh_pMt_pm1_pm2_pcm_P2(x_step(end,1:3), x_step(end,4:6),
285         params);
286     d_total = d_total + abs(P2(1));
287 end
288
289 % J_avg matches the optimizer cost: mean(norm(u)^2) over all timesteps
290 J_avg = J_sum / N_total;
291
292 % MCOT = J_avg / (M_total * g * d_per_step) -- energy metric per step
293 d_per_step = d_total / n;
294 mcot = J_avg / (M_total * g * d_per_step);
295 walking_speed = d_total / t_sim_total;
296
297 fprintf('\n== Mechanical Cost of Transport ==\n');
298 fprintf('J_avg = %.4f (mean norm(u)^2)\n', J_avg);
299 fprintf('Total dist = %.4f m\n', d_total);
300 fprintf('Step length = %.4f m\n', d_per_step);
301 fprintf('Total mass = %.1f kg\n', M_total);

```

```

301 fprintf('MCOT           = %.4f\n',    mcot);
302 fprintf('Walking speed= %.4f m/s\n', walking_speed);
303
304 end

```

Outputs

Mini_Project4_CW (command window output)

```

1 Step 1: q1_plus=0.3907, dq1_plus=-2.2327
2 Step#...1
3 Step 2: q1_plus=0.3907, dq1_plus=-2.2072
4 Step#...2
5 Step 3: q1_plus=0.3907, dq1_plus=-2.1927
6 Step#...3
7 Step 4: q1_plus=0.3907, dq1_plus=-2.1845
8 Step#...4
9 Step 5: q1_plus=0.3907, dq1_plus=-2.1801
10 Step#...5
11 Step 6: q1_plus=0.3907, dq1_plus=-2.1773
12 Step#...6
13 Step 7: q1_plus=0.3907, dq1_plus=-2.1758
14 Step#...7
15 Step 8: q1_plus=0.3907, dq1_plus=-2.1749
16 Step#...8
17 Step 9: q1_plus=0.3907, dq1_plus=-2.1744
18 Step#...9
19 Step 10: q1_plus=0.3907, dq1_plus=-2.1741
20 Step#...10
21 Step 11: q1_plus=0.3907, dq1_plus=-2.1739
22 Step#...11
23 Step 12: q1_plus=0.3907, dq1_plus=-2.1738
24 Step#...12
25 Step 13: q1_plus=0.3907, dq1_plus=-2.1738
26 Step#...13
27 Step 14: q1_plus=0.3907, dq1_plus=-2.1738
28 Step#...14
29 Step 15: q1_plus=0.3907, dq1_plus=-2.1738
30 Step#...15
31
32 === Mechanical Cost of Transport ===
33 Total dist   = 12.7806 m
34 Step length  = 0.8520 m
35 Total mass   = 35.0 kg
36 Walking speed= 1.7265 m/s
37 MCOT         = 1.1584

```

Simulation Plots

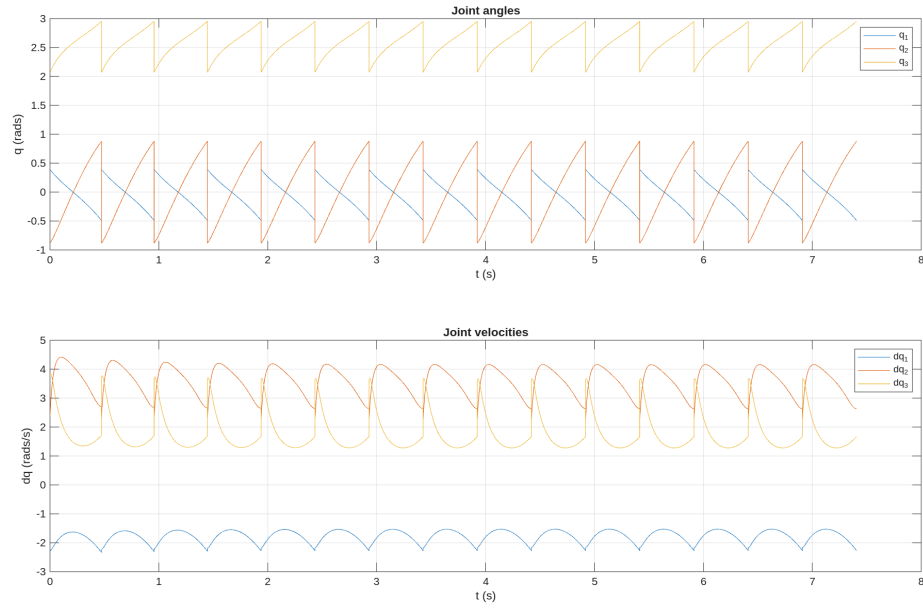


Figure 1: Joint angles and angular velocities.

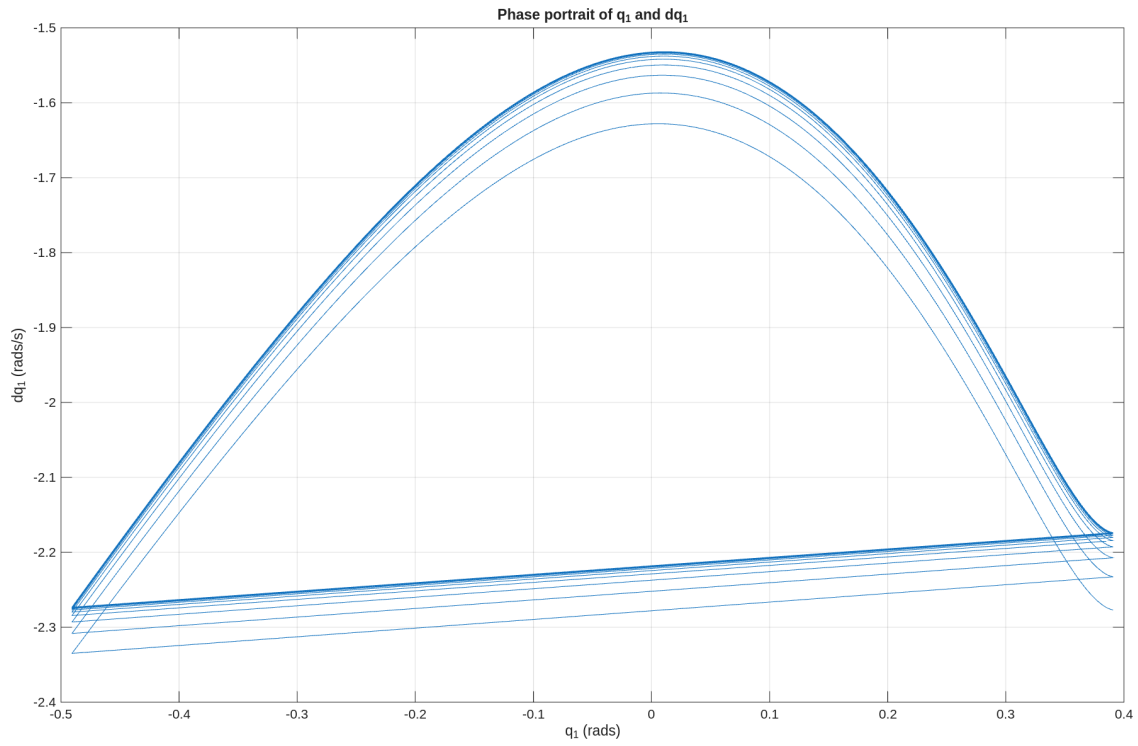


Figure 2: Phase portrait of the full dynamics (q_1 vs. $\dot{q}_1$).

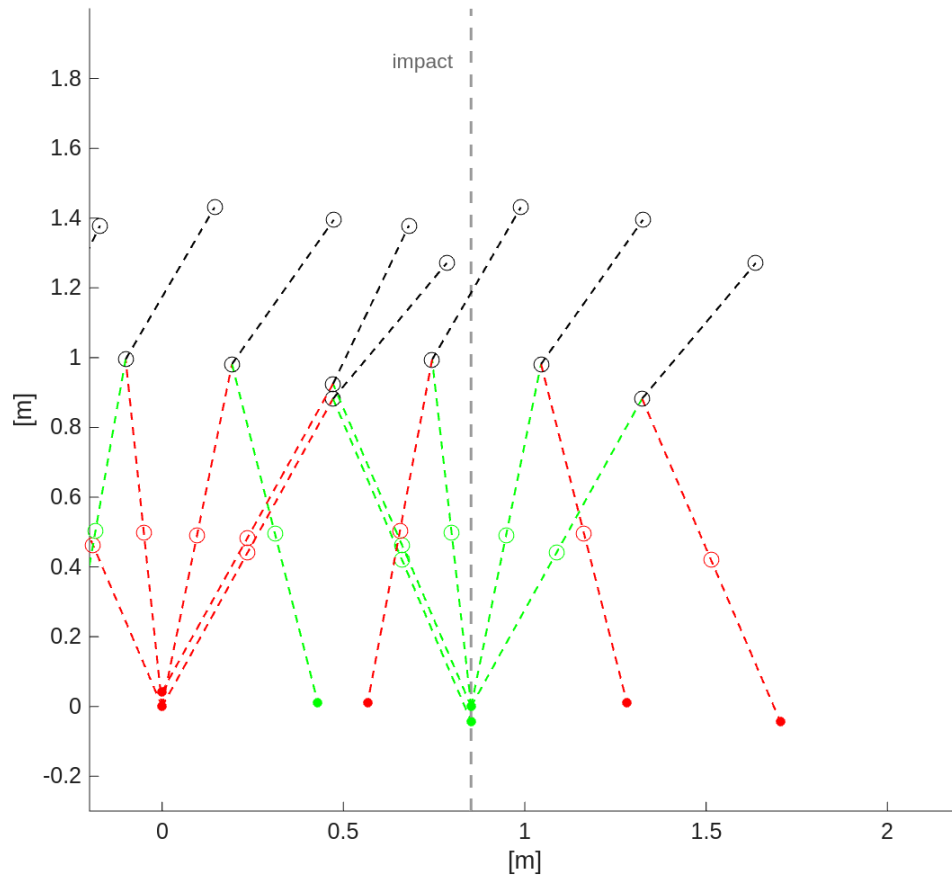


Figure 3: Frame-by-frame animation snapshot of the walking gait.

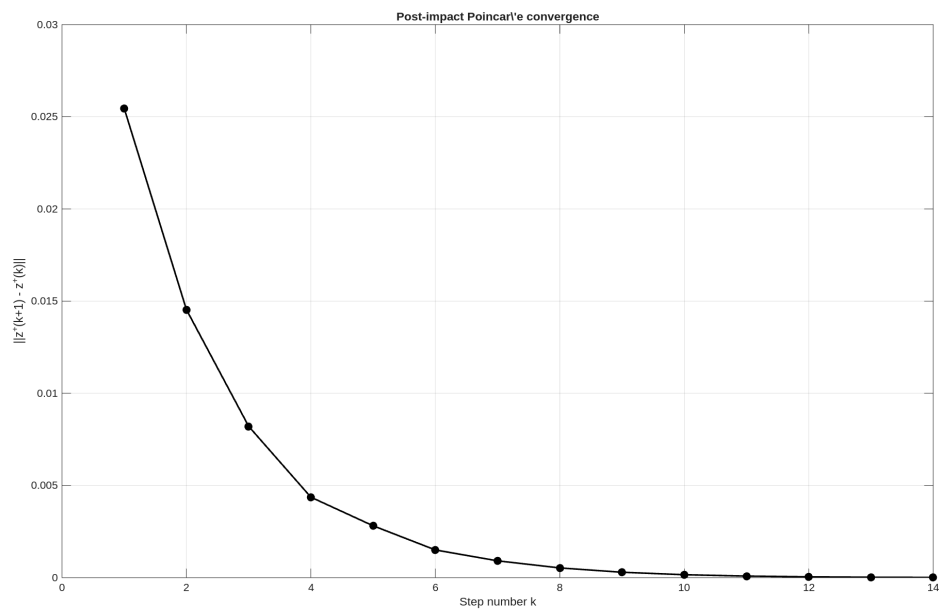


Figure 4: Step-to-step Poincaré map error showing convergence.